SOFTWARE ENGINEERING LAB - LAB 3

Component Modelling & Architectural Pattern Selection

Section H | Scenario: Self-Service Coffee Kiosk System

Student Name: Sanjit C | SRN: PES1UG24AM448 | Date: 12 November 2026

1. Objective & Scenario Overview

Objective: Systematically analyze candidate architectural styles, select the most appropriate pattern for a standalone Self-Service Coffee Kiosk System, and construct a rigorous UML Component Diagram with formal module boundaries, provided/required interfaces, hardware couplings, and transaction data flows.

Scenario Scope: A dedicated self-service coffee kiosk installed in a busy café. The system must fulfill four core functional capabilities: (1) allow customers to choose from 3 coffee types (Espresso, Americano, Latte), (2) support 2 drink sizes (Small, Large), (3) execute transactions exclusively through credit card processing, and (4) generate physical thermal receipts containing itemized order details. Technical constraints mandate handling interactive touchscreen inputs, interfacing with physical receipt printer hardware, and storing local menu and pricing information with zero dependency on continuous cloud connectivity.

2. Architectural Styles Comparative Analysis

Architectural Pattern	Structural Organization	Pros / Strengths	Cons / Trade-offs	Scenario Suitability
Layered Architecture (Selected)	Organized in distinct horizontal tiers: Presentation, Business Logic, and Hardware/Data Abstraction.	Clear separation of concerns; zero network overhead; sub-millisecond local method execution; clean hardware driver encapsulation.	Monolithic edge deployment; horizontal layer modifications require recompilation or binary packaging.	**Ideal: Dedicated edge hardware kiosk requiring instant tactile feedback and physical device control.**
Microservices Architecture	Decoupled, independently deployable micro-daemons communicating over HTTP/gRPC.	Independent scaling across clusters; polyglot language flexibility; localized process fault isolation.	Excessive operational complexity, high memory footprint, network latency, serialization serialization bottlenecks on edge CPU.	**Unsuitable:** Heavy architectural overhead and latency penalties on a single physical kiosk.
Client-Server Architecture	Thin touch client connecting over café Wi-Fi/LAN to a centralized in-store or cloud server.	Centralized menu management, centralized sales aggregation, simple client updates.	Network single-point-of-failure; Wi-Fi drops stall ordering queues; network latency harms touch responsiveness.	**Suboptimal:** Inherent dependency on network availability causes severe downtime risk.

3. Architectural Pattern Trade-off Evaluation

A Self-Service Coffee Kiosk is inherently an embedded, edge-computing device where user interactions and hardware operations occur synchronously in the physical world. Implementing Microservices introduces significant JSON serialization, local TCP port binding, and multi-process orchestration that offer no tangible benefit on a single CPU board. Similarly, a Client-Server topology exposes the café to catastrophic revenue loss whenever local Wi-Fi or ISP connections fail. Layered Architecture provides deterministic, low-latency execution, hardware driver modularity, and offline autonomy, making it the superior architectural choice.

3. Architecture Selection & Written Justification

Architecture Selection: "We chose Layered Architecture for the Self-Service Coffee Kiosk System."

1. Architectural Choice:

We selected a 3-Tier Layered Architecture structured into a **Presentation Layer** (Touchscreen GUI), a **Business Logic Layer** (Order Manager Component and Payment Service Component), and a **Hardware Abstraction & Persistence Layer** (Receipt Printer Driver, EMV Card Reader Interface, and Menu & Pricing Service). This architectural pattern establishes strict boundaries and dependency inversion, where higher-level business logic depends only on abstract interfaces provided by lower-level hardware drivers and data access components.

2. Two Scenario-Related Reasons for Selection:

• **Reason 1 — Hardware Abstraction & Peripheral Modularity:** The kiosk relies heavily on dedicated physical peripherals: a touch screen monitor, an ESC/POS thermal receipt printer, and an EMV credit card reader. The Layered pattern isolates device drivers into the lowest tier behind clean, well-defined interfaces (IReceiptPrinter and ICardReaderHardware). This completely decouples higher-level business logic (Order Manager) from vendor-specific serial/USB communication protocols. If the receipt printer runs out of paper or is replaced with a different hardware model, only the hardware driver tier is affected.

• **Reason 2 — Autonomous Edge Resilience & Offline Availability:** A busy café environment requires continuous, uninterrupted operation regardless of transient Wi-Fi drops or external network outages. Unlike a thin Client-Server model that halts ordering when backend connections fail, the Layered kiosk operates autonomously on local hardware. Menu browsing (Espresso, Americano, Latte in Small/Large), price calculations, and receipt formatting continue executing locally with zero latency and 100% kiosk uptime.

3. Security Advantage (PCI-DSS Compliance via Layered Isolation):

Because credit card is the exclusive payment method, strict PCI-DSS data protection is required. The Layered Architecture establishes a rigorous security boundary by encapsulating the Payment Service Component and EMV card hardware away from the Presentation Layer and Order Manager. Cardholder track data is processed and encrypted at the hardware/payment layer and is never transmitted to or logged in general memory. The Order Manager receives only sanitized authorization tokens through the narrow IPaymentProcessor interface, minimizing the system's attack surface and reducing audit scope.

4. Performance Benefit (Zero-Latency In-Memory Execution):

In a busy café with peak morning queues, customer turnaround time directly affects sales and satisfaction. Layered Architecture eliminates inter-service network serialization, TCP handshakes, and remote JSON parsing overhead inherent to Microservices. Inter-component method calls (Touchscreen UI → Order Manager → Menu Service) execute via sub-millisecond in-process invocations (< 2 ms). Rapid in-memory lookup of menu items and pricing ensures instantaneous 60 fps touch responsiveness, and receipts are dispatched to the printer immediately upon payment confirmation.

Summary Matrix of Architectural Benefits:

Evaluation Criterion	Layered Architecture Implementation	Direct Impact on Café Kiosk
Touchscreen Latency	In-memory event dispatching (< 2 ms)	Fluid 60 fps tactile feedback, zero UI lag
Payment Security	Isolated tier with tokenized responses	Strict PCI-DSS compliance; card data never leaks
Hardware Coupling	ESC/POS drivers encapsulated in bottom layer	Thermal printer / card reader hot-swappable
Operational Uptime	100% local autonomy on edge terminal	Zero dependence on café Wi-Fi for core ordering

4. Component Identification & Role Specification

In accordance with the chosen Layered Architecture, five modular software components and one hardware device were identified and specified:

Component Name	Layer / Tier	Core Responsibilities	Encapsulated State & Methods
Touchscreen UI	Presentation	Renders coffee items (Espresso, Americano, Latte), sizes (Small, Large); captures touch events; prompts payment; displays receipts.	+ displayMenu() + captureSelection() + promptPayment() + showConfirmation()
Order Manager Component (Core Business Logic)	Business Logic	Coordinates order lifecycle (Created -> PaymentPending -> Dispatched); calculates totals; orchestrates payment and printing.	- orderId: UUID - state: OrderState + createOrder() + calculateTotal() + triggerPayment() + dispatchPrint()
Payment Service Component (Security Gateway)	Business Logic / Security	Enforces PCI-DSS credit card transaction processing; interfaces with EMV terminal; encrypts card data; submits authorization requests.	- terminalId: String + initCardTxn() + captureCardData() + submitAuth() + verifyStatus()
Menu & Pricing Service	Data & Persistence	Stores and maintains catalog of coffee items, base prices, drink size multipliers; provides in-memory query access.	- menuStore: SQLite/Cache + getCatalog() + getPrice(type, size) + checkStockAvailability()
Receipt Printer Driver	Hardware Abstraction	Communicates with physical thermal printer over Serial/USB; formats ESC/POS byte-streams; monitors paper and device status.	- port: SerialPort - status: DeviceState + formatPayload() + transmitPrintJob() + checkPaperRoll() + cutPaper()
EMV Card Reader (Hardware Device)	Physical Hardware	Physical contactless NFC and EMV chip terminal handling card insertion, tap, and hardware Point-to-Point Encryption (P2PE).	- P2PE Secure Crypto Engine + promptTapOrInsert() + emitEncryptedBlob()

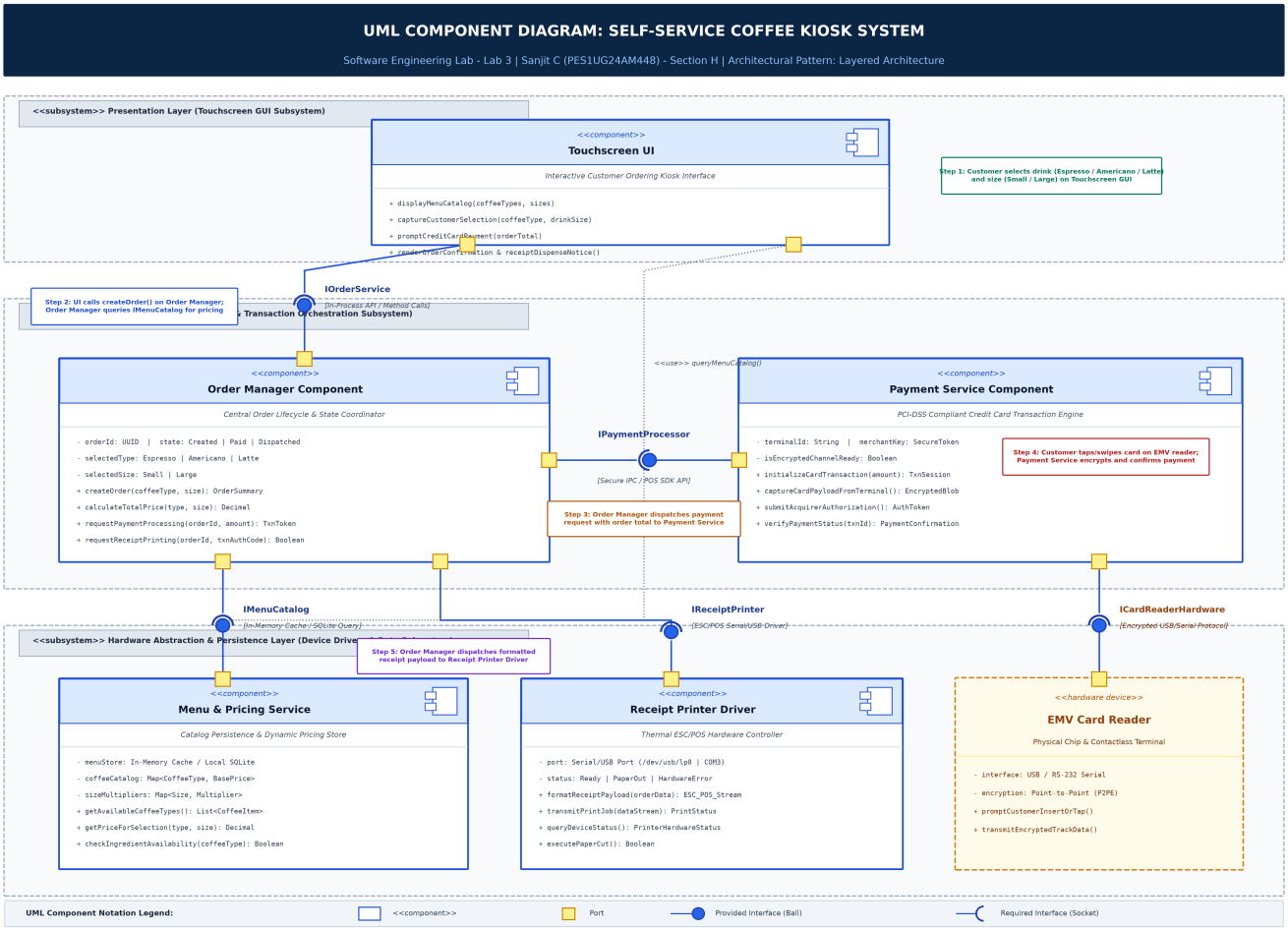
5. Component Interface Specifications

The system defines five explicit interfaces utilizing standard UML provided (ball) and required (socket) assembly connectors:

Interface Name	Provided By	Required By	Protocol / Mechanism	Methods & Data Flow
IOrderService	Order Manager Component	Touchscreen UI	In-Process API / Method Invocation	createOrder(type, size): OrderSummary cancelOrder(id): Void UI passes selections; receives order total and state.
IPaymentProcessor	Payment Service Component	Order Manager Component	Secure In-Process IPC / POS SDK	processPayment(orderId, amt): TxnToken getTxnStatus(id): TxnResult OM sends total; receives auth token & approval flag.
IMenuCatalog	Menu & Pricing Service	Order Manager & Touchscreen UI	In-Memory Direct Cache / Local SQLite	getCoffeeTypes(): List getPrice(type, size): Decimal Returns drink types, prices, and availability flags.
IReceiptPrinter	Receipt Printer Driver	Order Manager Component	ESC/POS Device Driver API	printReceipt(receiptData): PrintStatus getPaperStatus(): DeviceState Sends formatted text, timestamp, total, auth code.
ICardReaderHardware	EMV Card Reader Device	Payment Service Component	USB / RS-232 Encrypted Serial	readCardData(): EncryptedBlob promptSwipeTap(): Void Hardware transmits P2PE encrypted card payload.

6. UML Component Diagram: Self-Service Coffee Kiosk System

The diagram below illustrates the architectural components, ports, ball-and-socket assembly connectors, and subsystem boundaries:



7. Customer Transaction Execution Walkthrough (Step-by-Step)

• **Step 1 (Menu Selection):** Customer approaches the kiosk and taps the touchscreen. The **Touchscreen UI** queries IMenuCatalog on the **Menu & Pricing Service** to display the available coffee types (Espresso, Americano, Latte) and sizes (Small, Large) with current prices.

• **Step 2 (Order Creation):** Customer selects *Large Latte*. The UI invokes createOrder('Latte', 'Large') on IOrderService provided by the **Order Manager**. The Order Manager fetches price data ($4.75), generates a unique orderId, and transitions to PaymentPending.

• **Step 3 (Payment Dispatch):** The Order Manager invokes processPayment(orderId, $4.75) via IPaymentProcessor on the **Payment Service**. The UI prompts: *'Please tap or insert your credit card.'*

• **Step 4 (Card Authorization):** Customer taps card on the **EMV Card Reader**. The device passes P2PE encrypted card data over ICardReaderHardware to the Payment Service. The Payment Service validates with the payment gateway, confirms authorization, and returns an approval token to the Order Manager.

• **Step 5 (Receipt Printing):** Upon payment success, the Order Manager compiles order metadata (Date, Time, Order #, Large Latte, Total $4.75, Card Auth #) and dispatches it via IReceiptPrinter to the **Receipt Printer Driver**, which cuts and dispenses the receipt via ESC/POS commands. The UI displays order completion.

8. Deliverables & UML Compliance Verification

All lab requirements are fulfilled: (1) **At least 5 components** modeled (Touchscreen UI, Order Manager, Payment Service, Menu & Pricing Service, Receipt Printer Driver, plus EMV Reader device); (2) **At least 4 interfaces** demonstrated using official UML 2.0 ball (provided) and socket (required) assembly connectors; (3) Explicit ports on component boundaries with protocol labels; (4) Formal written justification covering two scenario reasons, PCI-DSS security, and sub-millisecond edge performance.